

Fitbit Data Analysis - Bellabeat Case Study (Task 3)

This documentation presents the analysis of Fitbit data as part of the Bellabeat Case Study. The task was to extract 5 facts from the dataset using a combination of SQL and Python (Pandas). The analysis leverages both SQLite queries and Pandas DataFrame operations for insights.

1. Setup and Data Loading

The analysis begins by loading Fitbit datasets (daily activity, sleep, and hourly steps) using Pandas and storing them in a local SQLite database for SQL-based queries.

```
18 def setup_database(daily_activity, sleep_data, hourly_steps):
19     """Create SQLite database and load data"""
20     # Remove existing database if it exists
21     if os.path.exists('fitbit_analysis.db'):
22         os.remove('fitbit_analysis.db')
23
24     # Create connection
25     conn = sqlite3.connect('fitbit_analysis.db')
26
27     # Load data into SQLite tables
28     daily_activity.to_sql('daily_activity_merged', conn, index=False, if_exists='replace')
29     sleep_data.to_sql('sleep_data_merged', conn, index=False, if_exists='replace')
30     hourly_steps.to_sql('hourly_steps_merged', conn, index=False, if_exists='replace')
31
32     print("✓ SQLite database created with Fitbit data")
33     return conn
34
35 def load_data():
36     """Load Fitbit datasets with pandas"""
37     daily_activity = pd.read_csv('mturkfitbit_dataset/Fitabase_Data/dailyActivity_merged.csv')
38     sleep_data = pd.read_csv('mturkfitbit_dataset/Fitabase_Data/sleepDay_merged.csv')
39     hourly_steps = pd.read_csv('mturkfitbit_dataset/Fitabase_Data/hourlySteps_merged.csv')
40
41     # Convert dates
42     daily_activity['ActivityDate'] = pd.to_datetime(daily_activity['ActivityDate'])
43     sleep_data['SleepDay'] = pd.to_datetime(sleep_data['SleepDay'])
44     hourly_steps['ActivityHour'] = pd.to_datetime(hourly_steps['ActivityHour'])
45
46     return daily_activity, sleep_data, hourly_steps
```

2. Fact 1: User Engagement Analysis

Using SQL, we count the number of days each user wore their Fitbit device and categorize them into Active, Moderate, or Light users. Pandas is used to summarize and calculate statistics.

```
40 def fact_1_user_engagement_sql(conn):
41     """FACT 1: User Engagement using SQL"""
42     print("\nFACT 1: USER ENGAGEMENT ANALYSIS")
43     print("\n"*50)
44
45     # SQL Query
46     sql_query = """
47     SELECT
48         Id,
49         COUNT(*) as DaysWorn,
50         CASE
51             WHEN COUNT(*) >= 25 THEN 'Active User (25+ days)'
52             WHEN COUNT(*) >= 15 THEN 'Moderate User (15-24 days)'
53             ELSE 'Light User (0-14 days)'
54         END as UserType
55     FROM daily_activity_merged
56     GROUP BY Id
57     ORDER BY DaysWorn DESC
58     """
59
60     # Execute SQL and get results as pandas DataFrame
61     result = pd.read_sql_query(sql_query, conn)
62
63     # Analysis using pandas
64     total_users = len(result)
65     active_users = len(result[result['UserType'] == 'Active User (25+ days)'])
66
67     # print(f"SQL Query: {sql_query.strip()}")
68     print(f"Results: {active_users}/{total_users} users ({active_users/total_users*100:.1f}%) wore Fitbit 25+ days")
69     print(f"Average days worn: {result['DaysWorn'].mean():.1f}")
70
71     return result
72
```

3. Fact 2: Activity Levels Analysis

SQL is used to compute the average steps and calories per user. Users are then classified into different activity levels based on their average daily steps. Pandas summarizes inactive and low-active users.

```
73 def fact_2_activity_levels_sql(conn):
74     """FACT 2: Activity Levels using SQL"""
75     print("\nFACT 2: ACTIVITY LEVELS ANALYSIS")
76     print("\n"*50)
77
78     # SQL Query
79     sql_query = """
80     SELECT
81         Id,
82         AVG(TotalSteps) as AvgSteps,
83         AVG(Calories) as AvgCalories,
84         CASE
85             WHEN AVG(TotalSteps) < 5000 THEN 'Inactive (<5,000 steps)'
86             WHEN AVG(TotalSteps) < 7500 THEN 'Low Active (5,000-7,499 steps)'
87             WHEN AVG(TotalSteps) < 10000 THEN 'Average Active (7,500-9,999 steps)'
88             WHEN AVG(TotalSteps) < 12500 THEN 'Active (10,000-12,499 steps)'
89             ELSE 'Very Active (12,500+ steps)'
90         END as ActivityLevel
91     FROM daily_activity_merged
92     GROUP BY Id
93     ORDER BY AvgSteps DESC
94     """
95
96     # Execute SQL and get results as pandas DataFrame
97     result = pd.read_sql_query(sql_query, conn)
98
99     # Analysis using pandas
100     inactive_low = len(result[result['ActivityLevel'].isin(['Inactive (<5,000 steps)', 'Low Active (5,000-7,499 steps)'])])
101     total_users = len(result)
102
103     # print(f"SQL Query: {sql_query.strip()}")
104     print(f"Results: {inactive_low}/{total_users} users ({inactive_low/total_users*100:.1f}%) are inactive/low-active")
105     print(f"Average steps per day: {result['AvgSteps'].mean():.0f}")
106
107     return result
108
```

4. Fact 3: Peak Activity Hours

Hourly step counts are aggregated using SQL to identify peak hours of physical activity. The top 5 hours with the most steps are reported.

```
109 def fact_3_peak_hours_sql(conn):
110     """FACT 3: Peak Activity Hours using SQL"""
111     print("\nFACT 3: PEAK ACTIVITY HOURS ANALYSIS")
112     print("="*50)
113
114     # SQL Query
115     sql_query = """
116     SELECT
117         CAST(strftime('%H', ActivityHour) AS INTEGER) as Hour,
118         SUM(StepTotal) as TotalSteps,
119         AVG(StepTotal) as AvgStepsPerUser
120     FROM hourly_steps_merged
121     GROUP BY Hour
122     ORDER BY TotalSteps DESC
123     LIMIT 5
124     """
125
126     # Execute SQL and get results as pandas DataFrame
127     result = pd.read_sql_query(sql_query, conn)
128
129     # Analysis using pandas
130     peak_hour = result.iloc[0]['Hour']
131
132     #print(f"SQL Query: {sql_query.strip()}")
133     print(f"Results: Peak activity at {peak_hour}:00")
134     print("Top 5 hours:")
135     for _, row in result.iterrows():
136         print(f" {row['Hour']}:00 - {row['TotalSteps']:}, total steps")
137
138     return result
```

5. Fact 4: Sleep and Activity Correlation

By joining daily activity and sleep data, SQL computes average sleep hours alongside steps and calories. Pandas is used to calculate correlation between sleep and activity and to summarize sleep categories.

```

140 def fact_4_sleep_correlation_sql(conn):
141     """FACT 4: Sleep and Activity Correlation using SQL"""
142     print("\nFACT 4: SLEEP AND ACTIVITY CORRELATION")
143     print("="*50)
144
145     # SQL Query
146     sql_query = """
147     SELECT
148         a.Id,
149         AVG(s.TotalMinutesAsleep/60.0) as AvgSleepHours,
150         AVG(a.TotalSteps) as AvgSteps,
151         AVG(a.Calories) as AvgCalories,
152         CASE
153             WHEN AVG(s.TotalMinutesAsleep/60.0) < 6 THEN 'Insufficient (<6 hours)'
154             WHEN AVG(s.TotalMinutesAsleep/60.0) < 7 THEN 'Below Recommended (6-7 hours)'
155             WHEN AVG(s.TotalMinutesAsleep/60.0) <= 9 THEN 'Adequate (7-9 hours)'
156             ELSE 'Excessive (>9 hours)'
157         END as SleepCategory
158     FROM daily_activity_merged a
159     INNER JOIN sleep_day_merged s ON a.Id = s.Id AND a.ActivityDate = s.SleepDay
160     GROUP BY a.Id
161     ORDER BY AvgSleepHours DESC
162     """
163
164     # Execute SQL and get results as pandas DataFrame
165     result = pd.read_sql_query(sql_query, conn)
166
167     # Analysis using pandas
168     correlation = result['AvgSleepHours'].corr(result['AvgSteps'])
169     sleep_counts = result['SleepCategory'].value_counts()
170
171     #print(f"SQL Query: {sql_query.strip()}")
172     print(f"Results: Sleep-activity correlation: {correlation:.3f}")
173     print("Sleep category distribution:")
174     for category, count in sleep_counts.items():
175         print(f"    {category}: {count} users")
176
177     return result

```

6. Fact 5: Weekly Activity Patterns

SQL is used to calculate average steps and calories by day of the week. Pandas highlights the most and least active days across the dataset.

```

179 def fact_5_weekly_patterns_sql(conn):
180     """FACT 5: Weekly Activity Patterns using SQL"""
181     print("\nFACT 5: WEEKLY ACTIVITY PATTERNS")
182     print("="*50)
183
184     # SQL Query
185     sql_query = """
186     SELECT
187         CAST(strftime('%w', ActivityDate) AS INTEGER) as DayOfWeek,
188         CASE CAST(strftime('%w', ActivityDate) AS INTEGER)
189             WHEN 0 THEN 'Sunday'
190             WHEN 1 THEN 'Monday'
191             WHEN 2 THEN 'Tuesday'
192             WHEN 3 THEN 'Wednesday'
193             WHEN 4 THEN 'Thursday'
194             WHEN 5 THEN 'Friday'
195             WHEN 6 THEN 'Saturday'
196         END as DayName,
197         AVG(TotalSteps) as AvgSteps,
198         AVG(Calories) as AvgCalories
199     FROM daily_activity_merged
200     GROUP BY DayOfWeek, DayName
201     ORDER BY DayOfWeek
202     """
203
204     # Execute SQL and get results as pandas DataFrame
205     result = pd.read_sql_query(sql_query, conn)
206     # Ctrl+L to chat Ctrl+K to generate
207     # Analysis using pandas
208     most_active = result.loc[result['AvgSteps'].idxmax(), 'DayName']
209     least_active = result.loc[result['AvgSteps'].idxmin(), 'DayName']
210
211     #print(f"SQL Query: {sql_query.strip()}")
212     print(f"Results: Most active: {most_active}, Least active: {least_active}")
213     print("Weekly averages:")
214     for _, row in result.iterrows():
215         print(f"    {row['DayName']}: {row['AvgSteps']:.0f} avg steps")
216
217     return result

```

7. Main Execution

The main() function orchestrates the workflow by loading data, setting up the database, executing SQL queries, and performing Pandas analysis.

```
219 def main():
220     """Main function using both SQL and Pandas"""
221     print("FITBIT DATA ANALYSIS - USING BOTH SQL AND PANDAS")
222     print("="*60)
223
224     # Load data with pandas
225     daily_activity, sleep_data, hourly_steps = load_data()
226     print(f"Data loaded: {len(daily_activity)} daily records, {len(sleep_data)} sleep records, {len(hourly_steps)} hourly records")
227
228     # Setup SQLite database
229     conn = setup_database(daily_activity, sleep_data, hourly_steps)
230
231
232     # Perform analysis using SQL queries with pandas for results processing
233     fact_1_user_engagement_sql(conn)
234     fact_2_activity_levels_sql(conn)
235     fact_3_peak_hours_sql(conn)
236     fact_4_sleep_correlation_sql(conn)
237     fact_5_weekly_patterns_sql(conn)
238
239     print("\n" + "="*60)
240     print("ANALYSIS COMPLETE!")
241     print("="*60)
242     print("✓ Used SQL queries to extract data")
243     print("✓ Used pandas to process and analyze results")
244     print("✓ Database file: fitbit_analysis.db")
245
246
247     conn.close()
248
```

Conclusion&Output

This analysis demonstrates the combined use of SQL and Pandas for data exploration and insight generation. The extracted facts provide a foundation for understanding user engagement, activity patterns, and the relationship between sleep and activity in Fitbit data.

OUTPUT:

```
FACT 1: USER ENGAGEMENT ANALYSIS
=====
Results: 29/33 users (87.9%) wore Fitbit 25+ days
Average days worn: 28.5

FACT 2: ACTIVITY LEVELS ANALYSIS
=====
Results: 17/33 users (51.5%) are inactive/low-active
Average steps per day: 7519

FACT 3: PEAK ACTIVITY HOURS ANALYSIS
=====
Results: Peak activity at 18:0:00
Top 5 hours:
 18:0:00 - 542,848.0 total steps
 19:0:00 - 528,552.0 total steps
 12:0:00 - 505,848.0 total steps
 17:0:00 - 498,511.0 total steps
 14:0:00 - 497,813.0 total steps

FACT 4: SLEEP AND ACTIVITY CORRELATION
=====
Results: Sleep-activity correlation: -0.064
Sleep category distribution:
Adequate (7-9 hours): 11 users
Insufficient (<6 hours): 8 users
Below Recommended (6-7 hours): 4 users
Excessive (>9 hours): 1 users

FACT 5: WEEKLY ACTIVITY PATTERNS
=====
Results: Most active: Saturday, Least active: Sunday
Weekly averages:
Sunday: 6933 avg steps
Monday: 7781 avg steps
Tuesday: 8125 avg steps
Wednesday: 7559 avg steps
```